RNN, LSTM and GRU

Praveen Kumar Chandaliya

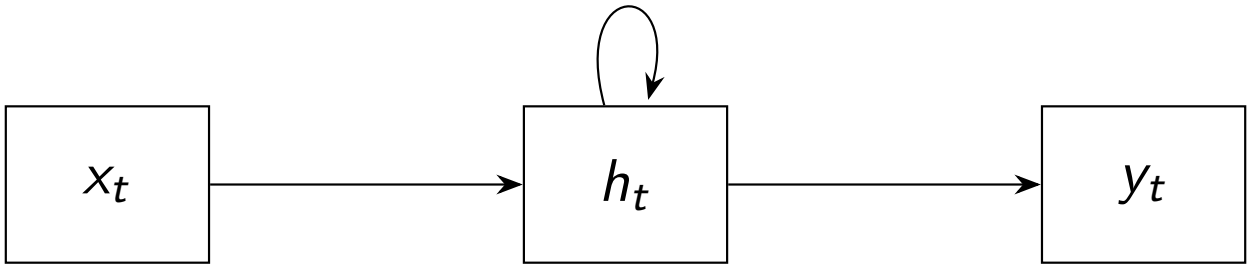
DoAI, SVNIT

February 25, 2026

Motivation: Why RNNs?

- Standard Neural Networks assume independent inputs.
- Many real-world tasks involve **sequential data**.
- Examples:
  - Language modelling
  - Speech recognition
  - Time-series forecasting
  - Machine translation
- RNNs introduce **memory** through hidden states.

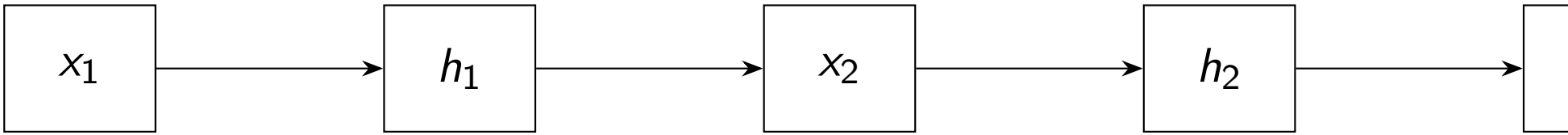
Recurrent Neural Network Architecture



$$h_t = f(W_{xh}x_t + W_{hh}h_{t-1} + b_h)$$

$$y_t = g(W_{hy}h_t + b_y)$$

Unfolded RNN Through Time



- Same weights reused at every time step.
- Enables modeling of long sequences.

RNN Forward Dynamics (Formal Definition)

Given sequence length T:

$$h_t = \phi(a_t)$$

$$a_t = W_{hh} h_{t-1} + W_{xh} x_t + b_h$$

$$y_t = g(W_{hy} h_t + b_y)$$

Total loss:

$$\mathcal{L} = \sum_{t=1}^{T} \mathcal{L}_t(y_t, \hat{y}_t)$$

Goal: Compute

$$\frac{\partial \mathcal{L}}{\partial W_{hh}}, \quad \frac{\partial \mathcal{L}}{\partial W_{xh}}, \quad \frac{\partial \mathcal{L}}{\partial W_{hy}}$$

Recursive Gradient Flow

Define:

$$\delta_t = \frac{\partial \mathcal{L}}{\partial h_t}$$

Using chain rule:

$$\frac{\partial \mathcal{L}}{\partial h_t} = \frac{\partial \mathcal{L}_t}{\partial h_t} + \frac{\partial \mathcal{L}}{\partial h_{t+1}} \frac{\partial h_{t+1}}{\partial h_t}$$

Thus,

$$\delta_t = \frac{\partial \mathcal{L}_t}{\partial h_t} + \delta_{t+1} \frac{\partial h_{t+1}}{\partial h_t}$$

Backward recursion starts from $t = T$.

Backpropagation Through Time (BPTT)

- RNNs are trained using **Backpropagation Through Time**.
- Gradients are computed across time steps.
- Total loss:

$$L = \sum_{t=1}^{T} L_t$$

- Gradients accumulate over time.

Vanishing and Exploding Gradients

- Repeated multiplication of gradients:

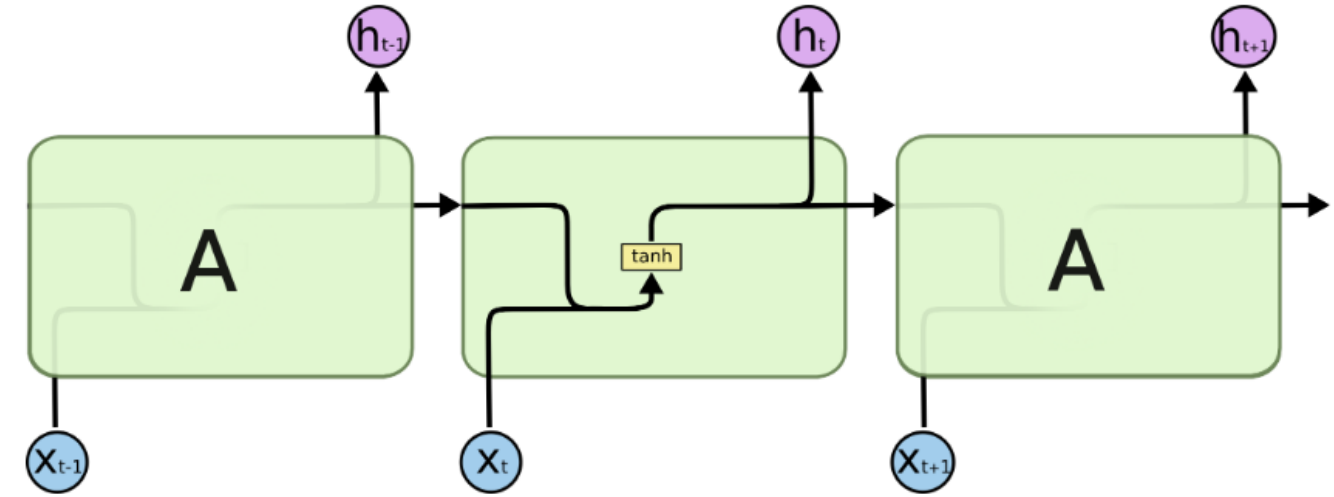
$$\frac{\partial L}{\partial h_t} = \prod_{k=t}^{T} \frac{\partial h_k}{\partial h_{k-1}}$$

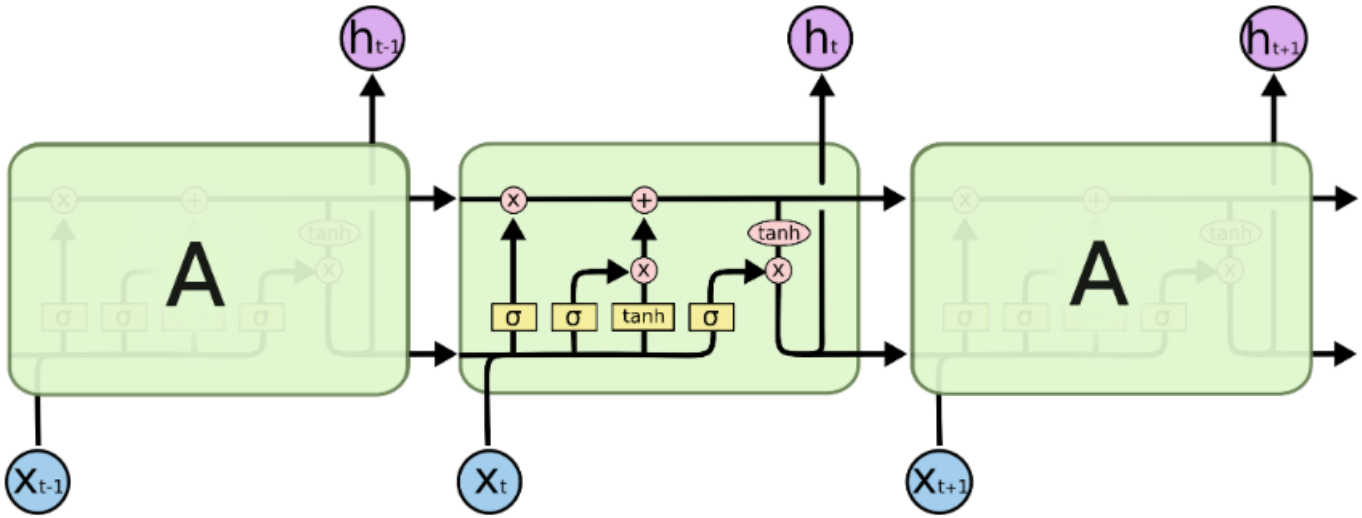
- If eigenvalues $< 1 \rightarrow$ Vanishing gradient
- If eigenvalues $> 1 \rightarrow$ Exploding gradient

Solutions:

- Gradient clipping
- Better initialization
- LSTM / GRU

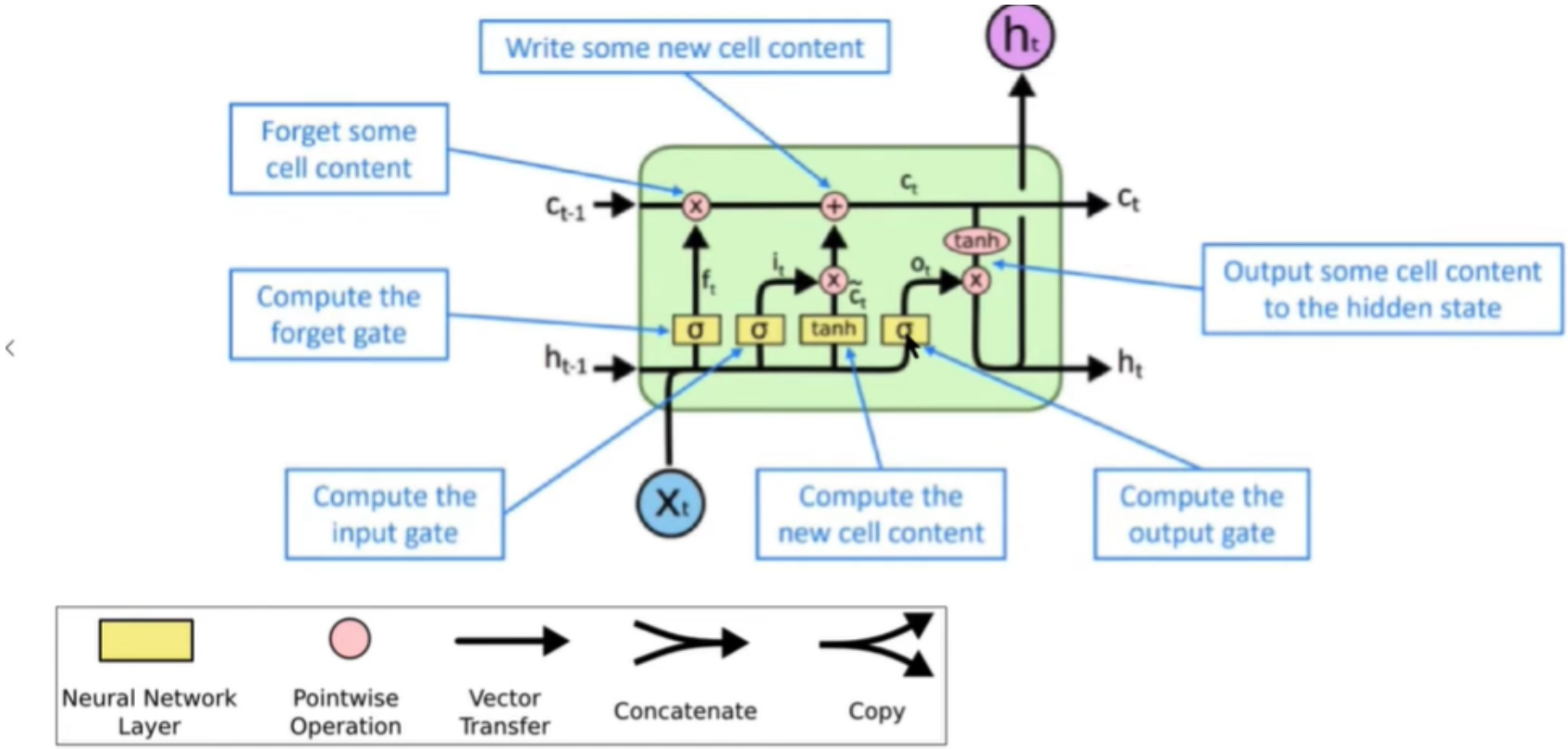
RNN vs LSTM


h_{t-1}
h_t
h_{t+1}
A
tanh
A
X_{t-1}
X_t
X_{t+1}


h_{t-1}
h_t
h_{t+1}
A
σ
σ
tanh
σ
tanh
A
X_{t-1}
X_t
X_{t+1}

Long Short-Term Memory (LSTM)[1]

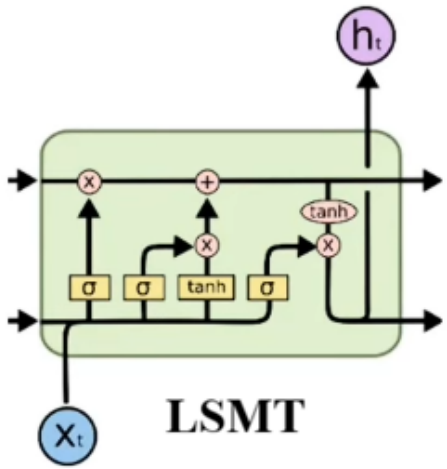
- Designed to overcome vanishing gradients and Introduces memory cell and gates.

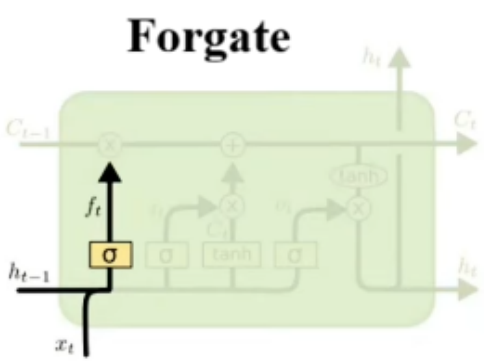


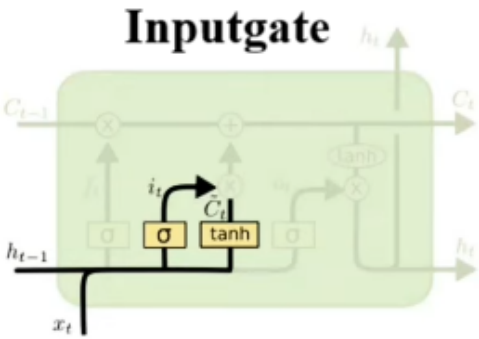
Credit: Christopher Manning, Stanford Univ

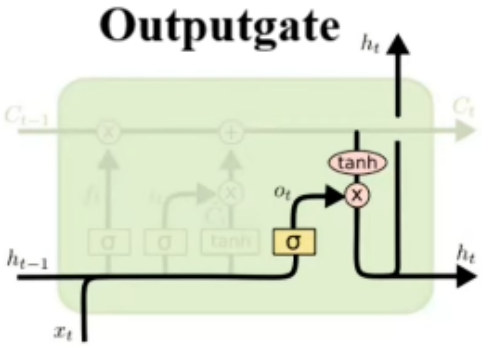
[1] https://colah.github.io/posts/2015-08-Understanding-LSTMs/

LSTM Gates[2]





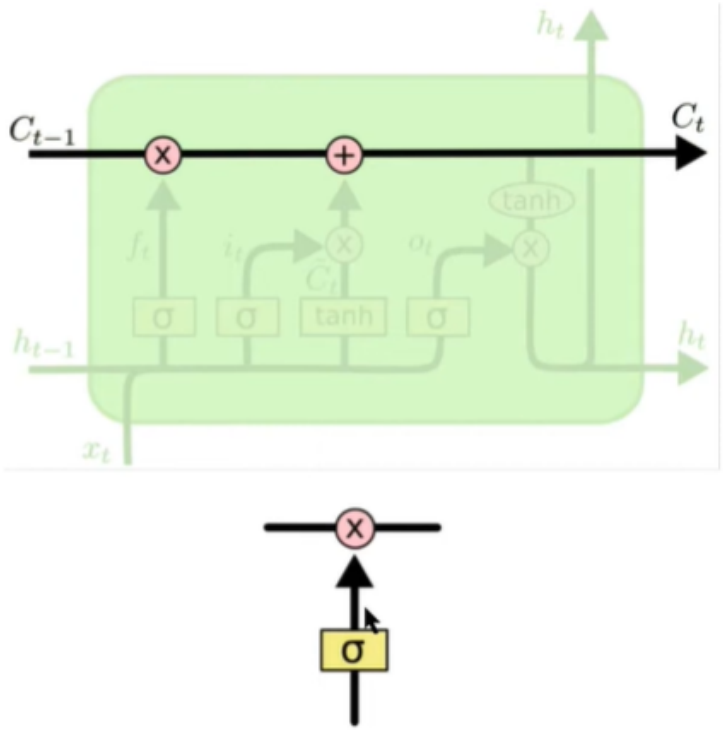




[2] https://www.youtube.com/watch?v=Akv3poqqwl4

LSTM Explained[3]

- **Cell state (C_t):**
  - Information can flow along cell state unchanged. Why is this important?
  - Ability to remove or add information to cell state, regulated by gates
- **Gates:**
  - Composed of a sigmoid neural net layer and a pointwise multiplication operation

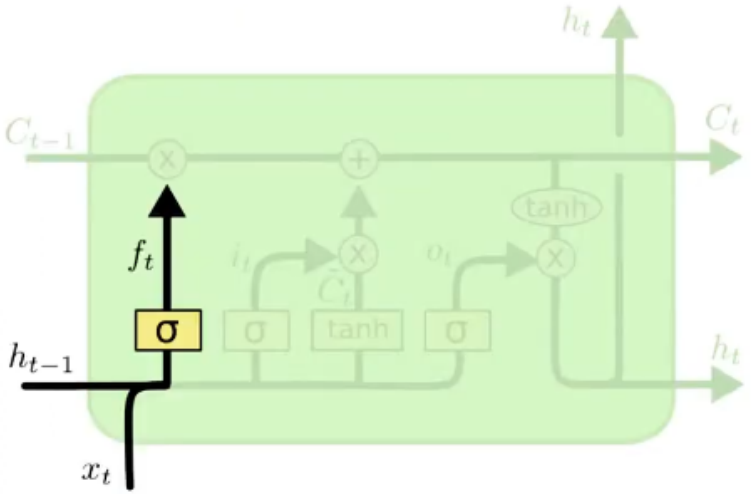


Credit: Christopher Olah

[3] https://www.youtube.com/watch?v=4vryB1QhBNc

LSMT: Forget gate

Forget gate: controls what is kept vs what is forgotten, from previous cell state

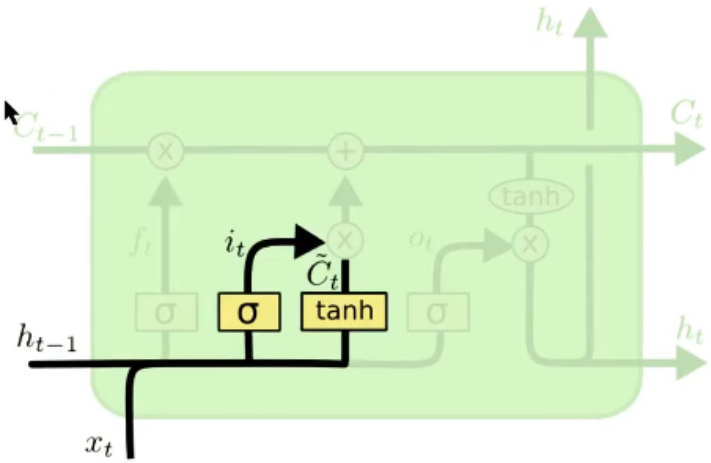


$$f_t = \sigma\left(W_f \cdot [h_{t-1}, x_t] \; + \; b_f\right)$$

Credit: Christopher Olah

LSMT: Input gate

Input gate: decides what information to throw away from the cell state
Cell content: new content to be written to cell



$$i_t = \sigma\left(W_i \cdot [h_{t-1}, x_t] \; + \; b_i\right)$$

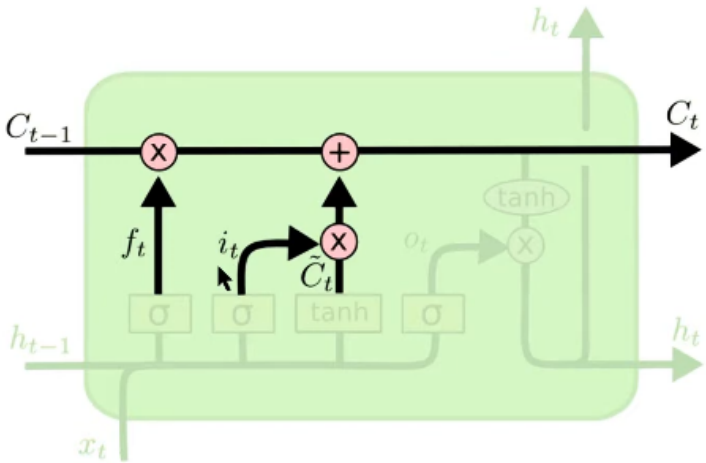
$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] \; + \; b_C)$$

Credit: Christopher Olah

Figure: Input gate: stage 1 and stage 2

LSMT: Cell gate

Cell state: erase ("forget") some content from last cell state, and write ("input") some new cell content



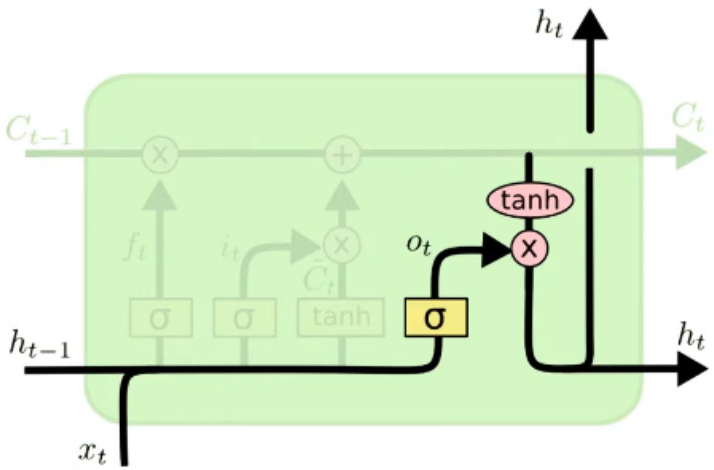
$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

Credit: Christopher Olah

Figure: Input gate: stage 3

LSMT: Output gate

Output gate: controls what parts of cell are output to hidden state
Hidden state: read ("output") some content from cell

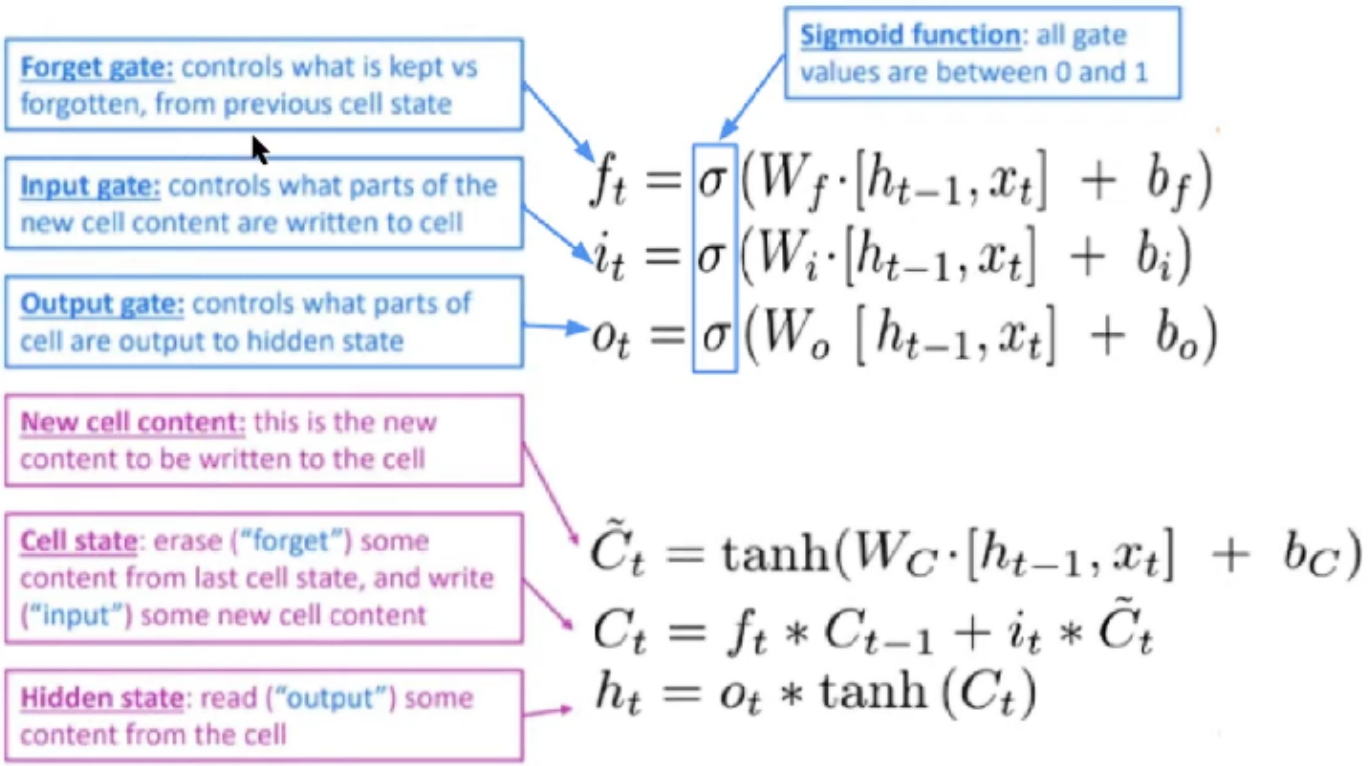


$$o_t = \sigma\left(W_o\left[h_{t-1}, x_t\right] + b_o\right)$$

$$h_t = o_t * \tanh\left(C_t\right)$$

Credit: *Christopher Olah*

Long Short-Term Memory



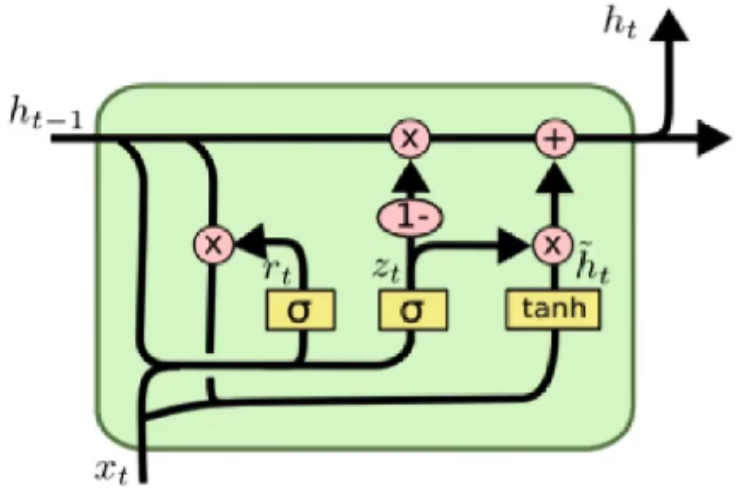
LSTM Visualization Resource

LSTM Animation

Interactive explanation of LSTM architecture:
Click here to view visualization

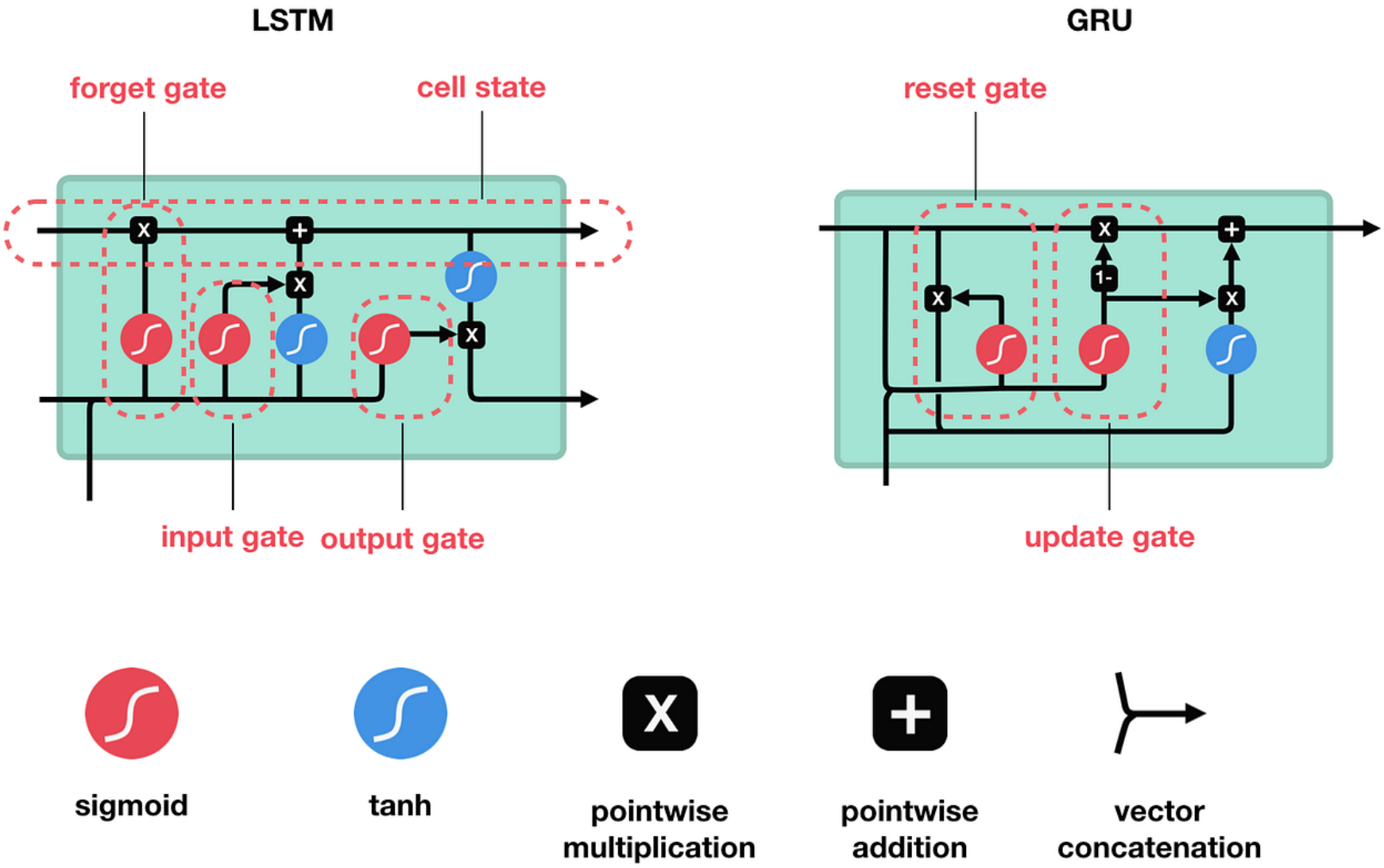
GRU: Gated Recurrent Neural Networks

- Proposed in 2014 as a simpler alternative to LSTM
- Combines forget and input gates into a single **update gate**
- Merges cell state and hidden state



[2] Chung et al. Empirical Evaluation of Gated Recurrent Neural Networks on Sequence Modeling. NeurIPS-W 2014

LSTM vs GRU


LSTM
forget gate
cell state
input gate
output gate
GRU
reset gate
update gate
1-
sigmoid
tanh
pointwise multiplication
pointwise addition
vector concatenation

GRU: Gated Recurrent Neural Networks

Update gate: controls what parts of hidden state are updated vs preserved

$$z_t = \sigma\left(W_z \cdot [h_{t-1}, x_t]\right)$$

Reset gate: controls what parts of previous hidden state are used to compute new content

$$r_t = \sigma\left(W_r \cdot [h_{t-1}, x_t]\right)$$

New hidden state content: reset gate selects useful parts of prev hidden state. Use this and current input to compute new hidden content.

$$\tilde{h}_t = \tanh\left(W \cdot [r_t * h_{t-1}, x_t]\right)$$

Hidden state: update gate simultaneously controls what is kept from previous hidden state, and what is updated to new hidden state content

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

LSTM vs GRU

- Input and forget gates of LSTMs are coupled by an update gate in GRUs; reset gate (GRUs) is applied directly to previous hidden state
- GRU has two gates, an LSTM has three gates; what does this tell you? **Lesser parameters to learn!**
- In GRUs:
  - No internal memory (c_t) different from exposed hidden state
  - No output gate as in LSTMs
- LSTM a good default choice (especially if data has long-range dependencies, or if training data is large); Switch to GRUs for speed and fewer parameters

Applications of RNNs, LSTM and GRU

- Language Modeling
- Handwritten Character Recognition
- Speech Recognition
- Video Analysis
- Time-Series Forecasting
- Machine Translation

Conclusion

- RNNs model sequential dependencies.
- BPTT enables training.
- LSTM/GRU solve gradient issues.
- Foundation for modern sequence models.